

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)
Кафедра математического моделирования

КУРСОВАЯ РАБОТА
АНАЛИЗ И ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ
СИСТЕМЫ ПО СБОРУ СТАТИСТИКИ С МАРКЕТПЛЕЙСОВ ДЛЯ
ПРОДАВЦОВ

Работу выполнил _____ В. А. Коломиец
(подпись)

Направление подготовки 02.03.03 Математическое обеспечение и
администрирование информационных систем

Направленность (профиль) Технологии разработки программных систем

Научный руководитель
канд. физ.-мат. наук, доц. _____ А. А. Евдокимов
(подпись, дата)

Нормоконтролер
канд. физ.-мат. наук, доц. _____ С.Е. Рубцов
(подпись, дата)

Краснодар

2025

РЕФЕРАТ

Курсовая работа содержит 28 с., 9 рис., 2 таблицы, 7 источников
ИНФОРМАЦИОННАЯ СИСТЕМА, БАЗЫ ДАННЫХ, БИЗНЕС-
ПРОЦЕССЫ, UML-ДИАГРАММЫ.

Целью курсовой работы является разработка информационной системы, которая помогает собирать статистику с разных маркетплейсов. Разрабатываемая информационная система необходима для анализа остатков товаров продавца и вывода результат в виде диаграмм, таблиц и графиков. Система должна автоматизировать такие процессы, как составление информации по объединённым товарам с разных маркетплейсов и вывод диаграмм для анализа.

В процессе выполнения данной работы проведен анализ требований заказчика и перевод их на технический язык.

СОДЕРЖАНИЕ

Введение	4
1 Начальное описание	5
2 Определение требований к создаваемой системе	6
2.1 Постановка задачи	7
2.2 Глоссарий предметной области	7
2.3 Нефункциональные требования к системе	9
2.4 Функциональные требования к системе	10
3 Создание вариантов использования	12
3.1 Добавление описаний вариантов использования	13
4 Проектировка базы данных	17
4.1 Создание базы данных	19
5 Проектирование функциональной структуры системы	21
5.1 Процесс «Получение данных от маркетплейсов»	22
5.2 Процесс «Сопоставление товаров»	23
5.3 Процесс «Формирование статистики»	25
Заключение	27
Список использованных источников	28

ВВЕДЕНИЕ

В современных условиях цифровая экономика играет все более значимую роль в жизни общества, охватывая такие сферы, как электронная коммерция, интернет-торговля и цифровые платформы. В этой области особенно выделяется сектор маркетплейсов, где функционируют такие крупные площадки, как Wildberries и Ozon. Для эффективного управления бизнесом на этих платформах необходимы квалифицированные специалисты и надежные программные решения, которые обеспечивают контроль за товарами, их позициями и статистикой продаж.

Для обеспечения высокого уровня эффективности управления товарами на маркетплейсах требуется специализированное программное обеспечение, способное собирать и анализировать данные из различных источников. Такие системы позволяют отслеживать позиции товаров, сравнивать цены, анализировать статистику продаж и обеспечивать прозрачность бизнес-процессов. Каждый продавец, работающий на нескольких платформах, нуждается в таких инструментах для принятия обоснованных решений.

Разработка программных решений для анализа данных о товарах на маркетплейсах является актуальной задачей, поскольку существующие альтернативы либо отсутствуют полностью, либо не удовлетворяют всем требованиям пользователей. В этой связи возникла необходимость создания системы, которая позволила бы объединять информацию о товарах с разных маркетплейсов в одном интерфейсе. Именно с этой целью и была выполнена настоящая курсовая работа.

1 Начальное описание

Перед разработчиком ставится задача создания веб-приложения для отслеживания товаров на маркетплейсах Wildberries и Ozon с возможностью объединения одинаковых товаров и отображения общей статистики. Предполагается, что данные о товарах собираются из различных маркетплейсов и хранятся с возможностью их последующего просмотра, анализа и сопоставления.

Сбор и хранение информации о товарах происходит следующим образом: перед внесением данных о товарах, указывается источник (идентификатор маркетплейса), откуда были получены сведения. Источник, именуемый далее типом магазина, будет определяться при добавлении учетных данных маркетплейса. Все магазины существуют внутри системы и связываются с вносимыми записями о товарах через свои учетные данные. В системе о магазине хранится следующая информация:

- тип (Wildberries или Ozon);
- API-токен;
- наименование.

Для каждого товара, полученного из маркетплейса, система хранит:

- уникальный идентификатор товара на маркетплейсе;
- наименование товара;
- цену;
- количество в наличии;
- ссылку на изображение;
- дату последнего обновления.

Пользователь может сопоставлять одинаковые товары из разных маркетплейсов, создавая связи между ними, что позволяет выводить

объединенную статистику по этим товарам. Система также позволяет просматривать и управлять этими сопоставлениями.

2 Определение требований к создаваемой системе

Определение требований – процесс, в ходе которого определяются задачи, поставленные перед разработчиками, и создаются модели, на основе которых планируется разработка системы. Требование – это условие, которому должно удовлетворять программное обеспечение, или свойство, которым оно должно обладать, чтобы удовлетворить потребность пользователя в решении некоторой задачи, или чтобы соответствовать условиям контракта, спецификации или стандарта [1]. Все требования делятся на функциональные и нефункциональные. Функциональные требования определяют действия, которые должна выполнять система, без учёта ограничений, связанных с её реализацией. Нефункциональные требования не определяют поведение системы, но описывают её свойства или атрибуты системного окружения. Например, нефункциональными являются требования к производительности системы и требования к безопасности хранения учетных данных [2].

Требования оформляются в виде ряда документов и моделей. К основным документам, согласно технологии разработки программного обеспечения, относятся: концепция, глоссарий предметной области, дополнительная спецификация [3]. Концепция определяет глобальные цели проекта и основные особенности разрабатываемой системы. Существенной частью концепции является постановка задачи разработки, определяющая требования к функциям системы [4]. Словарь предметной области (глоссарий) определяет общую терминологию для всех моделей и описаний требований к системе [5]. Дополнительная спецификация (технические требования) содержит описание нефункциональных требований к системе, таких, как надёжность, удобство использования, производительность, и другие [6].

2.1 Постановка задачи

Перед разработчиком ставится задача разработки клиент-серверного веб-приложения для отслеживания товаров на маркетплейсах Wildberries и Ozon с возможностью объединения одинаковых товаров и отображения общей статистики. Пользователем системы будет владелец или менеджер магазина на маркетплейсах.

Система должна позволять пользователям добавлять учетные данные своих магазинов на маркетплейсах, получать информацию о товарах через API, сопоставлять одинаковые товары из разных маркетплейсов и выводить объединенные данные о них. В системе может находиться информация о десятках или сотнях товаров. Пользователь может сохранять и удалять информацию о сопоставлениях товаров. Пользователи имеют доступ к системе в любое время, чтобы при необходимости получить актуальную информацию о своих товарах на разных маркетплейсах.

Данные о товарах выводятся в виде таблиц и графиков, которые можно использовать для анализа. Отчеты формируются автоматически на основе объединенных данных, предоставляемых системой.

Администратор системы может управлять учетными записями пользователей, но не имеет доступа к личным данным или учетным данным маркетплейсов пользователей.

2.2 Глоссарий предметной области

Глоссарий предметной области создаётся на основе описания системы отслеживания товаров на маркетплейсах и постановки задачи, а также терминологии, используемой при работе с API маркетплейсов. Глоссарий предназначен для описания терминологии области, в которой будет работать ПО. Выделяются термины, им даётся описание, рассчитанное на широкий круг

читателей (пользоваться этим описанием будут все лица, заинтересованные в разработке системы). Глоссарий составляется на русском языке и представляется в виде таблицы (таблица 1). Термины сопровождаются переводом на английский на тот случай, если термин будет использован в модели системы как название класса, пакета и т.п.

Таблица 1 – Глоссарий.

Маркетплейс (marketplace)	Платформа электронной коммерции, предоставляющая возможность продавцам размещать и продавать свои товары через интернет. В рамках данного проекта рассматриваются маркетплейсы Wildberries и Ozon.
Товар (product)	Наименование, реализуемое продавцом на маркетплейсе, характеризующееся уникальным ID, названием, ценой и количеством в наличии.
Сопоставление (mapping)	Процесс объединения информации о одинаковых товарах, реализуемых на разных маркетплейсах, с целью получения общей статистики.
API (Application Programming Interface)	Набор определений, протоколов и инструментов для создания программного обеспечения и приложений, позволяющий системам взаимодействовать между собой.
Магазин (store)	Коллекция товаров продавца на определенном маркетплейсе, идентифицируемая с помощью API-токена.
Токен (token)	Уникальный идентификатор, используемый для аутентификации при обращении к API маркетплейса или к системе.
Пользователь (user)	Физическое или юридическое лицо, использующее систему для отслеживания товаров на маркетплейсах.

2.3 Нефункциональные требования к системе

Дополнительная спецификация определяет нефункциональные требования к системе, такие, как надежность, удобство использования, производительность, а также ряд функциональных требований, являющихся общими для нескольких вариантов использования. Назначение дополнительных спецификаций – это определить требования к системе отслеживания товаров на маркетплейсах, которые не отражены в других документах и моделях. Вместе они образуют полный набор требований к системе. Рассмотрим дополнительную спецификацию:

Функциональные возможности

Система должна обеспечивать многопользовательский режим работы при этом, не ограничивая время использования, обеспечивать выполнение следующих правил:

- пользователь может добавить и сохранить учетные данные своего магазина на маркетплейсе.
- пользователь может удалять ранее добавленные учетные данные магазина.
- пользователь может сопоставлять и разъединять товары из разных маркетплейсов.
- система должна обновлять информацию о товарах по расписанию.

Требования по реализации

Система должна быть совместима с современными веб-браузерами (Chrome, Firefox, Safari, Edge).

Надёжность

Система должна быть в работоспособном состоянии 24 часа 7 дней в неделю, время простоя – не более 5%.

Производительность

Система должна поддерживать до 1000 одновременно работающих пользователей и обрабатывать запросы к API маркетплейсов в течение 30 секунд.

Проектные ограничения

Система должна обеспечивать шифрование API-токенов перед сохранением в базу данных и поддерживать протоколы безопасности при обмене данными.

2.4 Функциональные требования к системе

Функциональные требования к системе моделируются и документируются с помощью вариантов использования (use case). Вариант использования (use case) – связный элемент функциональности, предоставляемый системой при взаимодействии с действующими лицами, и соответствующий цели некоторого действующего лица. Действующее лицо (actor) – роль, обобщение элементов внешнего окружения системы, ведущих себя по отношению к системе одинаковым образом.

В контексте процесса управления требованиями варианты использования трактуются следующим образом:

- вариант использования описывает поведение системы при различных условиях, когда система отвечает на запрос одного из участников, называемого основным действующим лицом;
- основное действующее лицо инициирует взаимодействие с системой, чтобы добиться некоторой цели. Система отвечает, соблюдая интересы всех действующих лиц.

Модель вариантов использования состоит из диаграмм вариантов использования, текстовых описаний вариантов использования и диаграмм деятельности, моделирующих потоки событий вариантов использования. Диаграмма вариантов использования составляется на основе выявленных элементов модели, а затем устанавливаются связи между ними. Элементами

диаграмм вариантов использования являются варианты использования и действующие лица, соединённые разного рода связями.

3 Создание вариантов использования

Для того, чтобы корректно составить диаграмму вариантов использования нужно определить действующее лицо. Таковым является пользователь системы - владелец или менеджер магазина на маркетплейсах.

Далее выделим основные варианты использования, основываясь на работе и функционале, которые выполняет система. Так, например, сразу возникает вариант использования, повторяющийся не один раз, вариант – «Добавление магазина». Будем считать, что найдена "отправная точка" для выработки остальных вариантов использования. Таким образом, чтобы получить товары из маркетплейса, нужно сначала добавить учетные данные магазина – появляется вариант использования «Получение товаров». После получения товаров от разных маркетплейсов, пользователь может сопоставлять одинаковые товары – вариант использования «Сопоставление товаров». Кроме этих основных вариантов, пользователь может просматривать статистику по сопоставленным товарам – вариант использования «Просмотр статистики». В итоге, разместив на диаграмме необходимые объекты (актера и варианты использования), осталось их соединить и получить диаграмму, подобную рисунку 1.



Рисунок 1 – Диаграмма вариантов использования.

Для того, чтобы варианты использования были более понятны для дальнейшего построения системы, предлагается рассмотреть их описание.

3.1 Добавление описаний вариантов использования

Описания составляются специальным образом, чтобы уменьшить вероятность неверного толкования и облегчить восприятие текста. Каждое описание включает в себя:

- краткое описание, являющееся сжатым обзором варианта использования;
- основной поток событий, описывающий взаимодействие системы и действующих лиц, при котором достигается цель основного действующего лица;
- альтернативные потоки, описывающие обработку ошибок и исключительных ситуаций;
- подчинённые потоки, которые облегчают описание основного и альтернативных потоков;
- предусловия (которые могут отсутствовать) и постусловия (которые всегда есть).

Вариант использования «Добавление магазина»:

Краткое описание

Данный вариант использования описывает процесс добавления нового магазина на маркетплейсе в систему с использованием API-токена.

Основной поток событий:

Шаг 1. Пользователь запускает приложение и авторизуется в системе.

Шаг 2. Пользователь переходит на страницу добавления магазина.

Шаг 3. Система предлагает ввод данных.

Шаг 4. Пользователь выбирает тип маркетплейса (Wildberries или Ozon).

Шаг 5. Пользователь вводит API-токен маркетплейса.

Шаг 6. Система проверяет корректность токена, обращаясь к API маркетплейса.

Шаг 7. Система сохраняет зашифрованный токен в базе данных.

Альтернативные потоки

Ошибка шага 6 – некорректный токен:

Шаг 1. Система обнаруживает, что введенный токен некорректен или устарел.

Шаг 2. Система выдаёт сообщение об ошибке и предлагает пользователю ввести токен повторно.

Предусловия

Пользователь должен быть авторизован в системе.

Постусловия

Если вариант использования выполнен успешно, в системе появляется новый магазин с соответствующим типом и зашифрованным токеном, и система может получать данные о товарах из этого магазина.

Вариант использования «Получение товаров»:

Краткое описание:

Данный вариант использования позволяет системе получать информацию о товарах из маркетплейса посредством вызова соответствующего API.

Основной поток событий:

Шаг 1. Система выбирает один из добавленных магазинов пользователя.

Шаг 2. Система расшифровывает API-токен этого магазина.

Шаг 3. Система формирует запрос к API маркетплейса с использованием токена.

Шаг 4. Система получает и обрабатывает ответ от API.

Шаг 5. Система сохраняет информацию о товарах в базе данных.

Альтернативные потоки

Ошибка шага 3 – ошибка API:

Шаг 1. Система получает ошибку от API маркетплейса.

Шаг 2. Система записывает ошибку в лог и повторяет запрос через определенный интервал.

Предусловия

В системе должен быть добавлен хотя бы один магазин с корректным токеном.

Постусловия

Информация о товарах из маркетплейса сохраняется в базе данных и доступна для просмотра и сопоставления.

Вариант использования «Сопоставление товаров»:

Краткое описание:

Данный вариант использования позволяет пользователю сопоставлять одинаковые товары из разных маркетплейсов.

Основной поток событий:

Шаг 1. Пользователь просматривает список товаров из разных маркетплейсов.

Шаг 2. Пользователь выбирает товар из одного маркетплейса.

Шаг 3. Пользователь выбирает соответствующий ему товар из другого маркетплейса.

Шаг 4. Пользователь подтверждает сопоставление.

Шаг 5. Система создает связь между этими товарами в базе данных.

Альтернативные потоки

Ошибка шага 4 – товары уже сопоставлены:

Шаг 1. Система обнаруживает, что выбранные товары уже связаны друг с другом.

Шаг 2. Система уведомляет пользователя об этом и предлагает отменить существующее сопоставление.

Предусловия

В системе должны быть доступны товары как минимум из двух разных маркетплейсов.

Постусловия

Если вариант использования завершится успешно, товары будут связаны в базе данных, и по ним станет доступна объединенная статистика.

Вариант использования «Просмотр статистики»:

Краткое описание:

Данный вариант использования позволяет пользователю просматривать объединенную статистику по сопоставленным товарам.

Основной поток событий:

Шаг 1. Пользователь переходит на страницу статистики.

Шаг 2. Система отображает таблицы и графики с объединенной статистикой по сопоставленным товарам.

Шаг 3. Пользователь может фильтровать и сортировать данные.

Шаг 4. Пользователь может экспортировать данные в различные форматы.

Альтернативные потоки

Ошибка шага 2 – нет сопоставленных товаров:

Шаг 1. Система обнаруживает, что у пользователя нет сопоставленных товаров.

Шаг 2. Система уведомляет пользователя об этом и предлагает сопоставить товары.

Предусловия

Пользователь должен быть авторизован в системе и иметь хотя бы одно сопоставление товаров.

Постусловия

Пользователь получает доступ к объединенной статистике по сопоставленным товарам в удобном для анализа виде.

4 Проектировка базы данных

Перед созданием самой программы необходима база данных, к которой эта программа будет подключаться. Чтобы база данных была максимально корректна для начала нужно идентифицировать сущности, которые будут в ней существовать. Таких сущностей четыре: пользователь, магазин, товар и сопоставление.

Далее определим тип связей между сущностями и элементы, которые эти сущности содержат. Мы получим логическую модель вида, как представлено на рисунке 2.

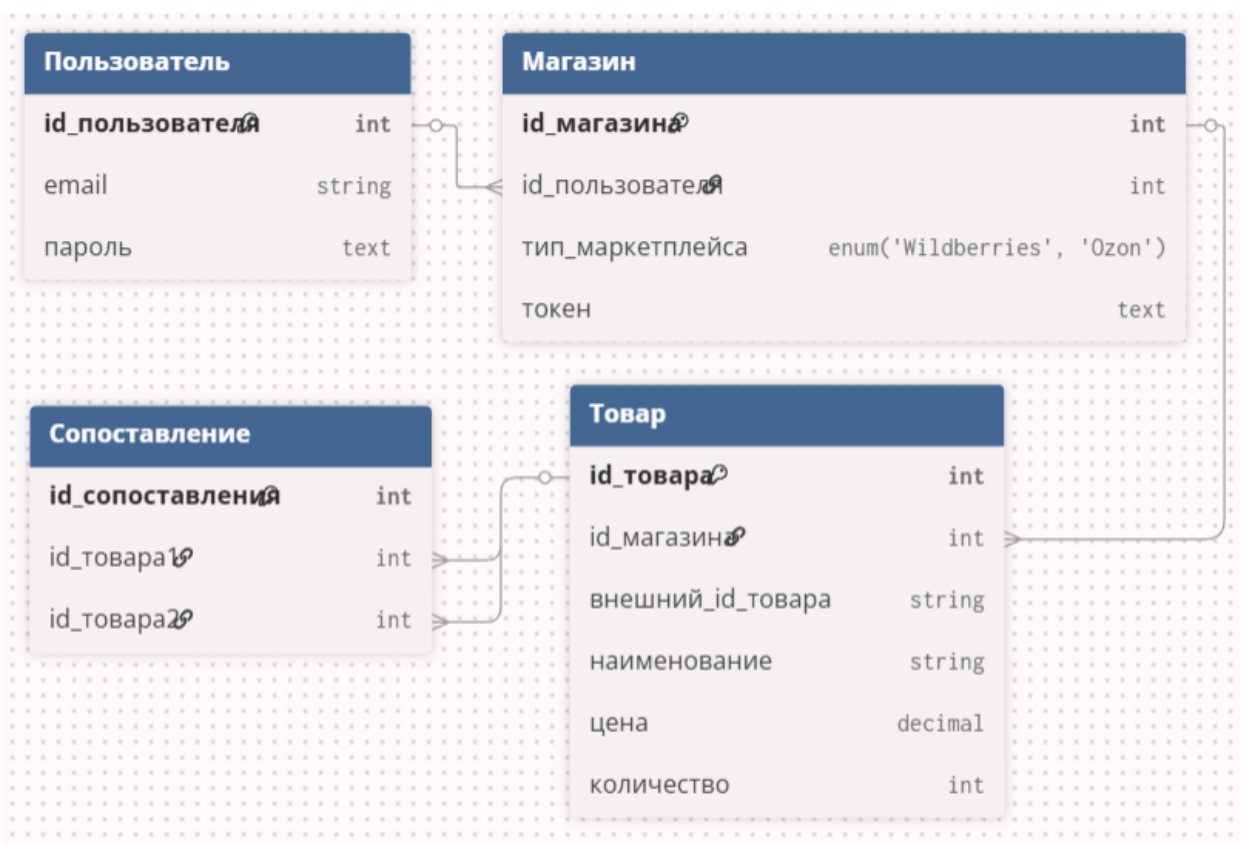


Рисунок 2 – Логическая модель.

Для каждой из сущностей были определены элементы, которые будут храниться в ней. Например, для сущности Пользователь элементами являются идентификатор, email и хэшированный пароль, которые содержат в себе

информацию, касающуюся конкретного пользователя. Элементы с приставкой id являются ключами, необходимыми для связей сущностей между собой. Составим физическую модель, рисунок 3, построенную на основании логической модели.

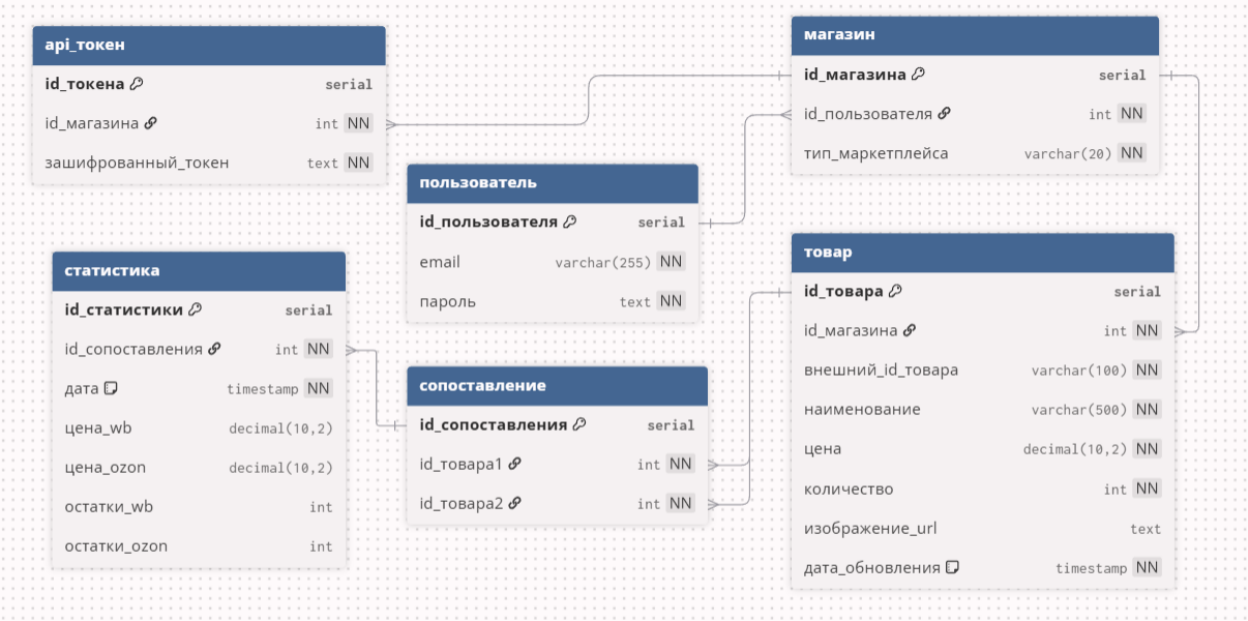


Рисунок 3 – Физическая модель данных.

На диаграмме появилась новая сущность - API токен. Она содержит в себе информацию об учетных данных для доступа к маркетплейсам. Чтобы о всех сущностях было правильное представление, физической модели необходимо подробное описание. Для этого создается таблица (таблица 2), где расписываются сущности, их описание и атрибуты.

Таблица 2. Описание данных

Сущность	Атрибуты	Описание
Справочные		
1. Пользователь	id_пользователя, email, пароль	Информация о пользователях системы

2. Магазин	id_магазина, id_пользователя, тип_маркетплейса, токен	Информация о магазинах пользователей на маркетплейсах
3. Товар	id_товара, id_магазина, внешний_id_товара, наименование, цена, количество	Информация о товарах в магазинах
4. Сопоставление	id_сопоставления, id_товара1, id_товара2	Информация о связях между одинаковыми товарами из разных маркетплейсов
5. API токен	id_токена, id_магазина, зашифрованный_токен	Информация об API-токенах магазинов
Оперативные		
6. Статистика	id_статистики, id_сопоставления, дата, цена_wb, цена_ozon, остатки_wb, остатки_ozon	Информация о статистике по сопоставленным товарам

Сущности делятся на справочные и оперативные. Справочные необходимы для хранения и вывода статичной информации, то есть той информации, которая не подвергается изменениям. Оперативные же предоставляют информацию, которая была в них занесена посредством использования программы [7].

4.1 Создание базы данных

После изучения и планирования структуры базы данных можно переходить к её созданию.

Для разработки была выбрана СУБД PostgreSQL по следующим причинам:

- Бесплатная объектно-реляционная СУБД с открытым исходным кодом;
- Наличие удобного интерфейса при работе с БД;
- Высокая мощность и широкая функциональность;
- Поддержка сложных запросов и масштабируемость.

Для создания таблиц используем команду CREATE TABLE с указанием всех атрибутов, ключей и связей. Таким образом получим SQL-скрипт создания структуры базы данных для приложения.

Все необходимые таблицы созданы, однако для полноценной работы справочные таблицы необходимо заполнить начальными данными. Для их заполнения используем команду INSERT INTO название_таблицы (элементы, в которые осуществляется запись) VALUES (записываемая информация для каждого элемента).

id_пользователя	email	пароль (хэш)
1	ivanov@example.com	\$2b\$12\$Kj5X2L3mQvT8nR9pWqZ4xOy1A7B6C5D4E3F2G1H0I
2	petrova@mail.ru	\$2b\$12\$LmN8oPqRsTuVwXyZaBcDeFgHiJkLmNoPqRsTuVwXy
3	smirnov1985@gmail.com	\$2b\$12\$AbCdEfGhIjKlMnOpQrStUvWxYz123456789012345
4	ekaterina.ozon@yandex.ru	\$2b\$12\$XyZaBcDeFgHiJkLmNoPqRsTuVwXyZaBcDeFgHiJkL
5	admin@wbstats.local	\$2b\$12\$QwErTyUiOpAsDfGhJkLzXcVbNmQwErTyUiOpAsDfG

Рисунок 4 – Данные из таблицы.

Таким образом, получена готовая база данных, к которой в дальнейшем будет подключено веб-приложение.

5 Проектирование функциональной структуры системы

Для разработки качественной системы необходимо детально рассмотреть её работу. Выделить входные и выходные данные, параметры, отвечающие за работу внутри системы, и атрибуты напрямую или косвенно выполняющие эту самую работу. Таким образом, получена диаграмма, представленная на рисунке 5.

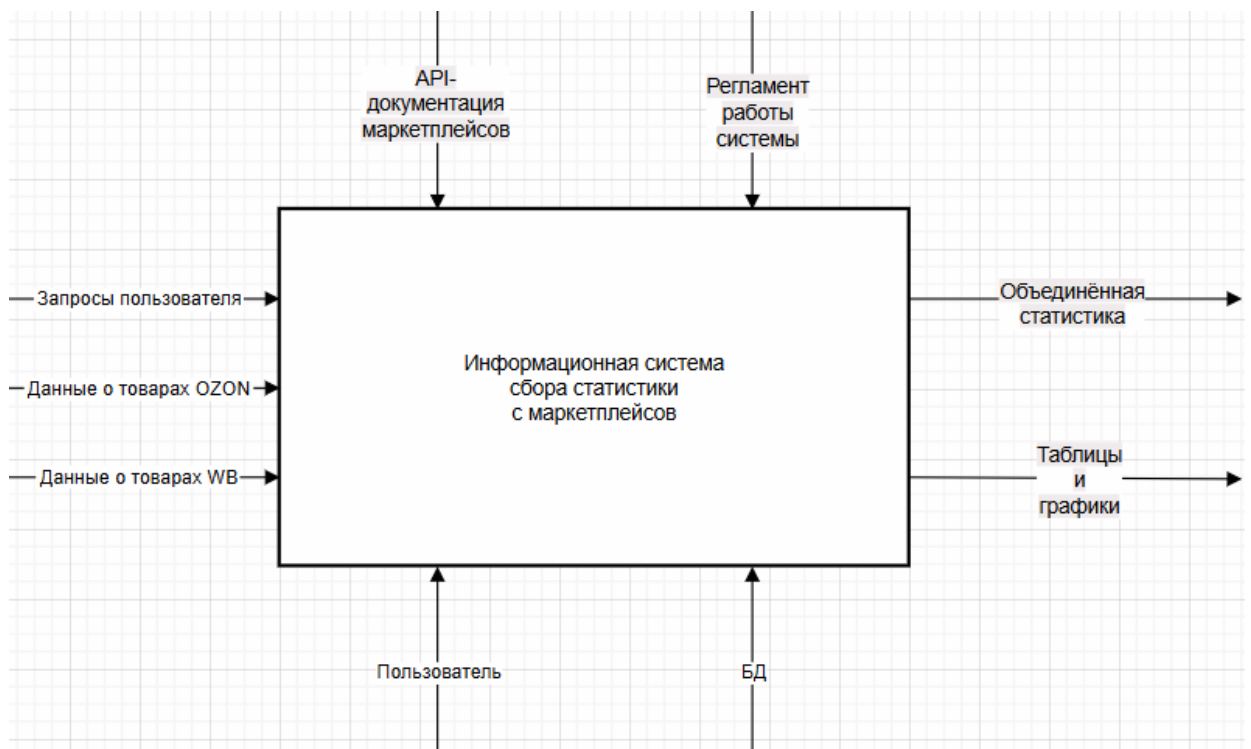


Рисунок 5 – Контекстная диаграмма.

Ориентируясь на контекстную диаграмму, можно точно задать потоки входа и выхода в программе. Однако такой диаграммы недостаточно, чтобы полностью распланировать весь ход действий программы и её шаги. Чтобы узнать, какие этапы в себя включает работа программы, декомпозируем её до уровня диаграммы, как на рисунке 6.

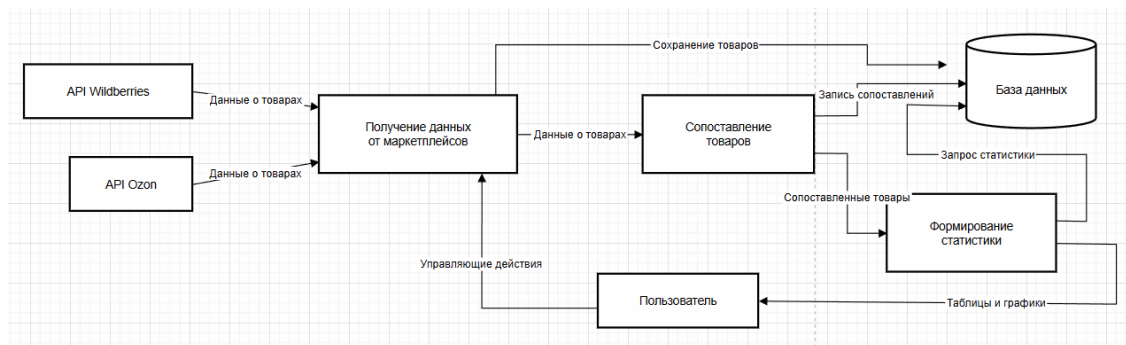


Рисунок 6 – Декомпозиция контекстной диаграммы.

Теперь работа программы разбита по этапам. Так получено, что система включает в себя три основных процесса: получение данных от маркетплейсов, сопоставление товаров и формирование статистики. У каждого из них есть свои входные и выходные данные, которые передаются или приходят от предыдущего процесса или извне.

5.1 Процесс «Получение данных от маркетплейсов»

Стоит детально рассмотреть отдельно каждый из процессов и разобраться в каждом потоке данных, включенных в них. Для начала рассмотрим процесс получения данных от маркетплейсов, построив соответствующую диаграмму на рисунке 7.

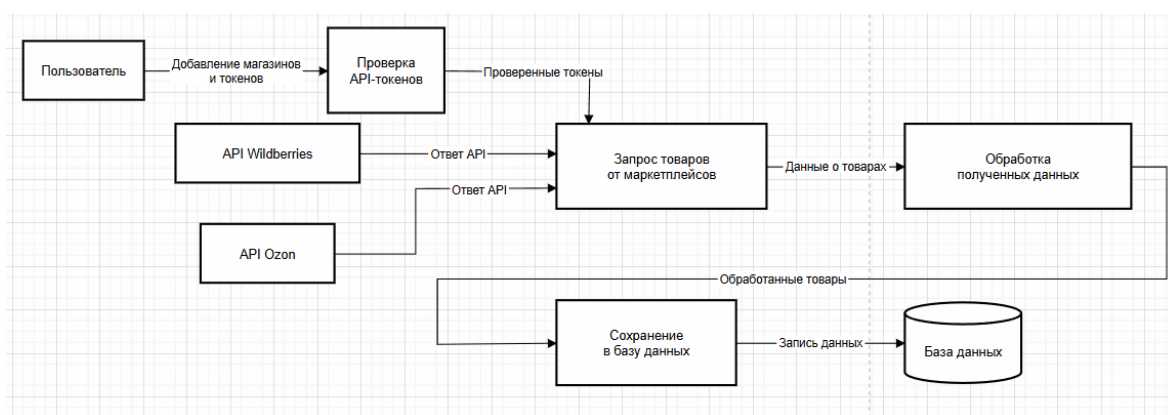


Рисунок 7 – Декомпозиция процесса «Получение данных от маркетплейсов».

Получение данных от маркетплейсов состоит из проверки API-токенов, запроса товаров от маркетплейсов, обработки полученных данных и сохранения в базу данных. Разберемся с каждым этапом по отдельности.

Начнем с процесса «Проверка API-токенов». На вход поступают данные о добавленных магазинах, где проверяется корректность и актуальность API-токенов. Далее токены направляются (поток «Проверенные токены») в процесс «Запрос товаров от маркетплейсов». Этот процесс регулируется API-документацией маркетплейсов, а также сопровождается действиями Пользователя, который добавляет учетные данные своих магазинов.

Процесс «Запрос товаров от маркетплейсов» начинается с прихода к нему потока «Проверенные токены», после чего процесс обращается к API маркетплейсов и получает ответ, который может быть передан дальше в систему. Все эти действия регулируются API-документацией маркетплейсов и сопровождаются деятельностью Пользователя, который может инициировать обновление данных.

От ранее рассмотренных процессов существует ещё несколько связанных с ними процессов. «Обработка полученных данных» включает в себя анализ полученной информации от маркетплейсов, например, наименование товаров, цены и остатки, и регулируется структурой ответов от API, передаёт поток данных «Обработанные товары» к следующему процессу.

Процесс «Сохранение в базу данных» принимает входной поток «Обработанные товары» и сохраняет их в соответствующие таблицы, направляя поток к другим процессам системы.

5.2 Процесс «Сопоставление товаров»

После «Получения данных от маркетплейсов» следует процесс «Сопоставление товаров», для которого была составлена диаграмма, аналогичная предыдущей, представленная на рисунке 8.

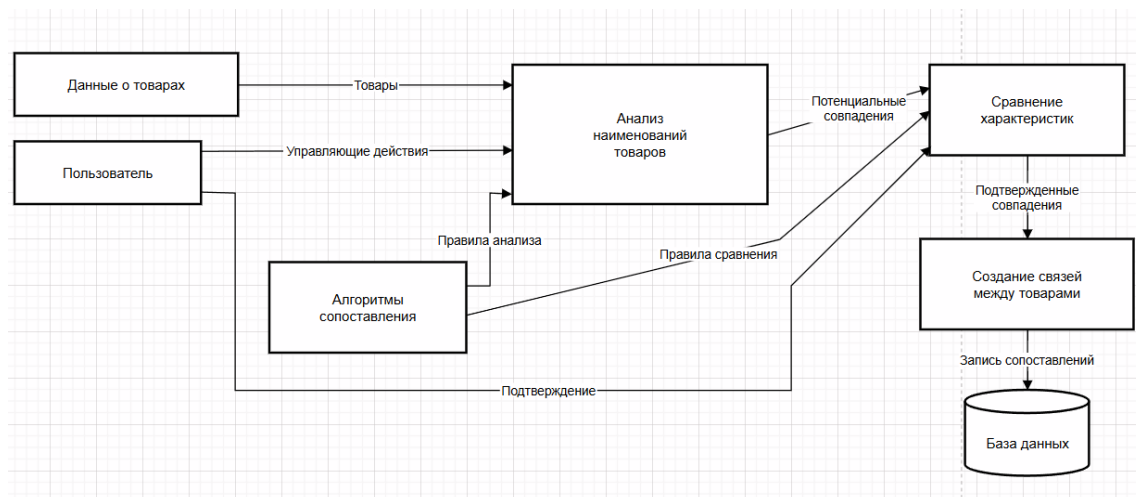


Рисунок 8 – Декомпозиция процесса «Сопоставление товаров».

Сопоставление товаров состоит из трёх этапов: анализ наименований товаров, сравнение характеристик, создание связей между товарами.

Начнем с процесса «Анализ наименований товаров». На вход поступают данные о товарах, которые пришли от процесса «Получение данных от маркетплейсов». Данные анализируются с целью нахождения потенциальных совпадений по наименованию, артикулу или описанию, и результат этого анализа направляется в виде потока «Потенциальные совпадения» в следующий процесс.

Процесс «Сравнение характеристик» начинается с прихода к нему потока «Потенциальные совпадения», после чего производится сравнение других параметров товаров (цены, изображения, артикулы и т.д.), и формируется выходной поток «Подтвержденные совпадения», который направляется к следующему этапу. Этот и предыдущий процессы регулируются алгоритмами сопоставления, а также сопровождаются работой в системе пользователей.

Процесс «Создание связей между товарами» выполняет роль объединения информации о товарах из разных маркетплейсов, формируя записи в таблице сопоставлений, и включает в себя работу с базой данных (БД).

5.3 Процесс «Формирование статистики»

Осталось рассмотреть последний из трёх процессов – процесс «Формирование статистики». Для него также была построена и представлена на рисунке 8 диаграмма декомпозиции.

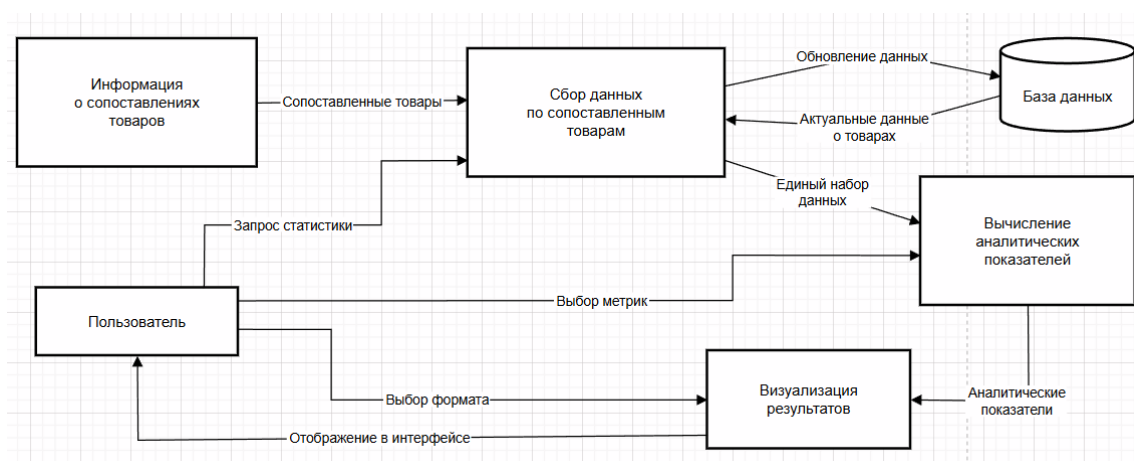


Рисунок 9 – Декомпозиция процесса «Формирование статистики».

Формирование статистики включает в себя три основных этапа: сбор данных по сопоставленным товарам, вычисление аналитических показателей и визуализация результатов.

Процесс «Сбор данных по сопоставленным товарам» начинается с получения информации о сопоставлениях товаров, которые были созданы на предыдущем этапе. Система собирает актуальные данные о ценах, остатках и других характеристиках товаров из разных маркетплейсов, формируя единый набор данных для анализа. Этот процесс сопровождается регулярным обновлением информации и контролем целостности данных.

Процесс «Вычисление аналитических показателей» принимает единый набор данных и производит вычисления, такие как разница в ценах, соотношение остатков, динамика изменений и другие метрики, позволяющие пользователю принимать обоснованные решения по управлению товарами на маркетплейсах.

Процесс «Визуализация результатов» преобразует вычисленные аналитические показатели в наглядные таблицы, графики и диаграммы, которые отображаются в интерфейсе приложения для удобства анализа и принятия решений.

ЗАКЛЮЧЕНИЕ

В рамках выполнения курсовой работы была спроектирована информационная система для сбора, анализа и визуализации данных о товарах с маркетплейсов Wildberries и Ozon. Основной целью проекта являлось проектирование удобного инструмента, позволяющего продавцам объединять информацию об одинаковых товарах из разных площадок и получать общую статистику для принятия обоснованных управленческих решений.

В процессе работы были решены следующие задачи:

- проведён анализ предметной области и требований заказчика;
- определены функциональные и нефункциональные требования к системе;
- разработана концептуальная модель, включая постановку задачи, глоссарий и дополнительные спецификации;
- спроектирована база данных (логическая и физическая модели, SQL-скрипты);
- создана функциональная структура системы с использованием диаграмм потоков данных (DFD): контекстная диаграмма, декомпозиция основных процессов и детализация каждого из них;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. IEEE Std 830-1998. Рекомендуемая практика по спецификациям требований к программному обеспечению. IEEE Computer Society, 1998. (Перевод на русский: <https://github.com/maxvipon/IEEE-Std-830-1998-RU/blob/master/IEEE%20STD%20830-1998%20%28RU%29.md>)
2. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. 3-е изд. М.: Русская Редакция, 2014. 704 с.
3. Халл Э., Джексон К., Дик Дж. Инженерия требований. 3-е изд. М.: ДМК Пресс, 2015. 608 с. ISBN 978-5-97060-214-0.
4. Соммервилл И. Инженерия программного обеспечения. 10-е изд. М.: Вильямс, 2019. 800 с. ISBN 978-5-907144-10-1.
5. Леффингуэлл Д., Уидриг Д. Управление требованиями. 3-е изд. М.: ДМК Пресс, 2014. 544 с. ISBN 978-5-97060-213-7.
6. Липаев В.В. Инженерия требований к программному обеспечению. М.: СИНТЕГ, 2012. 432 с. ISBN 978-5-89638-134-1.
7. Шамшин Ю. Введение в базы данных. Лекции. 2017. – URL: https://dbs.academy.lv/lection/2017_db_04_mod.pdf (дата обращения: 21.12.2025).